Cours de préparation au projet

Ce que voit un détecteur de musique IA

Signal audio, codecs et générateurs neuronaux, artefacts, détection, évaluation — ce qu'il faut maîtriser pour réaliser le projet

Mode d'emploi. Ce cours remplace les cours de notions N1–N27 du dossier : il se lit dans l'ordre (environ 8 h de lecture, exercices compris, estimation personnelle ; laboratoires L1–L9 hors lecture, résumés en annexe C). Il suppose un niveau M2 IA (CNN, Transformers, PyTorch, ROC) et *aucun* cours d'audio. Chaque notion porte son identifiant ([N12]) pour retrouver le dossier. Ce qui vient d'un article est présenté comme *établi par l'article* ; ce qui est une conjecture du projet est signalé *(hypothèse du projet : ainsi)*.
Statut des faits. Les étiquettes suivent la convention du dossier : [V] page ou article ouvert pendant la préparation de ce cours (mais lu par extraction automatique du texte : *relire le PDF avant de citer un chiffre*) ; [R] extrait de recherche, résumé ou fiche de dépôt, article non lu en entier ; [M] connaissance générale non revérifiée, à contrôler avant de s'y fier ; [N] source non ouverte (page refusée, vidéo, accès restreint). Sans étiquette : calcul ou raisonnement donné dans le cours, que vous pouvez refaire. Les points à revérifier sont regroupés en annexe D.

Section	Notions du dossier	Semaine du projet	Page
1 État de l'art	N17, N18, N19 (cadre), N24	avant la semaine 1	1
2 Signal audio	N1, N2, N3, N4, N5	phase 0, semaine 1	3
3 Chaîne commune et raccourcis	N19, N20, N27	semaines 1–2	6
4 Apprentissage et évaluation	N6, N7, N8, N9, N26	semaine 3	7
5 Génération audio neuronale	N10–N16, N25, N12	semaines 1–2	9
6 Détecteurs M0–M4	N17–N22	semaines 3–4	12
7 Robustesse	N21	semaine 4	14
8 Données, méthode, éthique	N23, N24, N27	toutes	15

Annexes : A sources (p. 16), B exercices et corrigés (p. 17), C laboratoires et parcours (p. 17), D points à revérifier (p. 18), E aide-mémoire (p. 18), F glossaire (p. 18).

1 Le problème et l'état de l'art

1.1 Ce que fait un détecteur, et ce qui est difficile

Un *détecteur* est une fonction $s(x) \in \mathbb{R}$ qui associe à un extrait audio x un score de « tendance au faux » ; on décide « faux » quand $s(x) > \tau$, où τ est un seuil. On l'entraîne sur des exemples réels (étiquette 0) et faux (étiquette 1). Trois difficultés structurent tout le projet :

- la **généralisation** : un détecteur appris sur les sorties d'un générateur détecte-t-il celles d'un autre, jamais vu ?
- la **robustesse** : que devient-il quand on change la vitesse, la hauteur, qu'on compresse en MP3 ?
- les **raccourcis** (*shortcuts*) : le détecteur lit-il la « musique générée » ou un signe sans rapport (format, fréquence d'échantillonnage, durée, silence) ?

Deux familles de méthodes existent : les détecteurs *appris* sur spectrogrammes (CNN, Transformers) et les détecteurs par *artefact explicite* (empreintes spectrales, *fakeprints*). Le projet compare des représentants des deux.

1.2 Les sources, une par une

Afchar, Hennequin, Meseguer-Brocal (Deezer) — « Detecting music deepfakes is easy but actually hard » (arXiv 2405.04181).[V] Les auteurs partent de FMA « medium » (environ 25 000 pistes) et font passer chaque piste réelle par neuf décodeurs d'auto-encodeurs : EnCodec à 3, 6 et 24 kbit/s, DAC (version LAC) à 2, 7 et 14 kbit/s, le décodeur de Musika, et une reconstruction par spectrogramme mel et Griffin-Lim (256 et 512 bandes), soit 249 820 pistes, rééchantillonnées à 44,1 kHz. Un CNN de six couches convolutives (environ 1,6 M de paramètres), appliqué à des extraits aléatoires de 0,8 s, atteint 99,8 % d'exactitude en distribution ; le spectrogramme d'*amplitude* donne les meilleurs résultats. En revanche, le transfert est bon *au sein d'une famille* de décodeurs (EnCodec 24 → 6 et 3 kbit/s) et quasi nul *entre familles* ; l'exactitude s'effondre sous décalage de hauteur, bruit blanc et ré-encodage MP3/AAC/Opus à 64 kbit/s, mais résiste à l'étirement temporel, l'égalisation et la réverbération ; la plupart des scores sont proches de 0 ou 1, et le modèle

bascule vers « réel » sous perturbation. **C'est le précédent le plus proche du projet** : notre apport est de tester sur de *vrais générateurs* et de mesurer les raccourcis.

Afchar et al. — « A Fourier Explanation of AI-music Artifacts » (ISMIR 2025, arXiv 2506.19108).[R] L'article analyse les modules de déconvolution (suréchantillonnage) des générateurs et montre que leurs sorties portent de petits pics spectraux systématiques, apparentés au *checkerboard* connu en vision ; ces pics dépendent de l'*architecture* choisie, non des données d'entraînement ni des poids appris. Il en tire un critère de détection simple et interprétable (selon un résumé tiers, une régression logistique sur ces empreintes) dont l'exactitude dépasse 99 % dans plusieurs scénarios, y compris sur des modèles ouverts (EnCodec, DAC, Musika) et sur Suno et Udio. La robustesse aux manipulations (rééchantillonnage, décalage de hauteur) est laissée à de futurs travaux. C'est la base théorique de H1 (§5) et du détecteur M1.

Comanducci, Bestagini, Tubaro — « FakeMusicCaps » (arXiv 2409.10684).[V] Le jeu recrée MusicCaps (environ 5,5 k extraits de 10 s, chacun avec une légende écrite par un musicien) : chaque légende sert de requête à cinq modèles texte-vers-musique — MusicGen (checkpoint medium, ≈1,5 Md de paramètres), MusicLDM, AudioLDM2 (version musique), Stable Audio Open, Mustango — ce qui donne 27 605 pistes, ≈77 h. Format : 10 s, **mono, 16 kHz, flottant 32 bits, WAV** (les sorties stéréo sont mélangées en mono). Les auteurs entraînent trois détecteurs de référence (M5, RawNet2, ResNet18 sur spectrogramme) sur six classes (cinq générateurs plus MusicCaps réel) et rapportent par exemple 1,00 d'exactitude pour ResNet18 en ensemble fermé, mais seulement 0,85 en ensemble ouvert (seuil sur le rapport des deux meilleurs scores) ; ils jugent eux-mêmes ces résultats préliminaires. Le test ne compte que 320 pistes et il n'est pas précisé si l'archive contient aussi les extraits réels de MusicCaps (point à vérifier, annexe D).

Han, Wang, Gui — « MusicDET » (ICML 2026, arXiv 2605.18072).[V] Détecteur *zéro-coup* : il n'apprend que sur de la musique réelle (flots normalisants sur spectrogramme d'énergie, extraits de 4 s à 16 kHz mono) et signale comme « IA » ce qui a une faible vraisemblance. Sur FakeMusicCaps il obtient un EER moyen de 4,51 %, contre plus de 11 % pour les détecteurs discriminatifs comparés (le meilleur, W2V2-AASIST, 11,46 % à la dernière lecture ; les valeurs lues auparavant divergent, annexe D). Mais l'EER monte à environ 44,7 % sous décalage de hauteur, 44,1 % sous bruit blanc et 41,8 % sous MP3. L'esprit « entraîné sans faux générateur » est voisin du projet ; la robustesse y est le point faible déclaré.

Dugelay et al. — « Improved Robustness in AI-Generated Music Detection » (ISMIR 2026, arXiv 2607.27454).[V] Les détecteurs actuels s'effondrent sous changement de vitesse (par exemple le détecteur d'Afchar et al. tombe à F1 ≈0,008 sur Suno v5 en test). La méthode transforme l'empreinte spectrale sur un axe *logarithmique* de fréquence, où une vitesse devient une translation, puis applique un *unique filtre de corrélation croisée appris* suivi d'un *max-pooling* : l'invariance est structurelle. Détails et chiffres en §6–7. L'article ne teste ni MP3 ni bruit, et cite l'égalisation, la compression de dynamique et les codecs comme non testés.

Park et al. — « Probing Token Spaces under Generator Shift in AI-Generated Music Detection » (arXiv 2606.08663, atelier ML4Audio d'ICML 2026).[R] Étude contrôlée où le classifieur est fixe et seul change l'espace de jetons audio (jetons de codecs, jetons dérivés d'un modèle auto-supervisé). Sur MoM-open (reconstruction ouverte dont le réel vient de FMA et MTG-Jamendo), les découpages standards sont presque saturés, mais restreindre les *sources de faux* révèle de grands écarts entre espaces de jetons : le choix du tokenizer est présenté comme un axe expérimental principal. Il n'existe donc pas de « meilleur » espace universel (résumé tiers : X-Codec domine sur Udio, des jetons dérivés de MERT sur Suno v3.5).

Oh — « ArtifactNet » (arXiv 2604.16254).[V] Détecteur léger fondé sur l'hypothèse que la quantification par RVQ perd de l'information de façon irréversible, ce qui laisse un « résidu » de structure différente dans l'audio généré. Point crucial pour notre H3 : avant un entraînement tenant compte des codecs, le modèle classait comme IA **98,7 %** des pistes réelles FMA en MP3 ; le taux tombe à 8,0 % après entraînement conscient des codecs. C'est une démonstration directe du raccourci « format ». L'auteur note aussi que ses attributs supposent du 44,1 kHz (à 16 kHz, le signal faiblit).

SONICS et AI-OpenBMAT (dossier).[V] SONICS (arXiv 2408.14080) : environ 97 000 titres dont plus de 49 000 générés par Suno et Udio ; ses checkpoints SpecTTTra (code et modèles sous licence MIT) servent de

référence externe *en inférence seule* (M4). AI-OpenBMAT (arXiv 2602.06823) : 3 294 extraits d'une minute imitant la télévision ; en diffusion télévisée, les détecteurs testés ne dépassent pas environ 61 % de F1 global (SpecTTTra ; 28 % pour un CNN) et sont plus faibles encore quand la musique est en arrière-plan (accès et licence à vérifier).

1.3 Trois leçons pour le projet

1. **Les scores en distribution ne disent presque rien** : 99,8 % (Afchar), 1,00 (FakeMusicCaps), partitions « saturées » (Park et al.). La question est le *transfert*.
2. **Le format fuit** : FMA est en MP3, FakeMusicCaps en WAV 16 kHz mono de 10 s, et un détecteur peut lire cette différence (ArtifactNet : 98,7 % de faux positifs sur FMA). D'où la chaîne commune et le détecteur M0 (§3).
3. **La robustesse est fragile et se mesure mal** : AUC et F1 peuvent diverger (§7) ; l'invariance à la vitesse est structurelle pour M3, mais rien n'est établi pour MP3 ni bruit.

1.4 Le projet en une figure

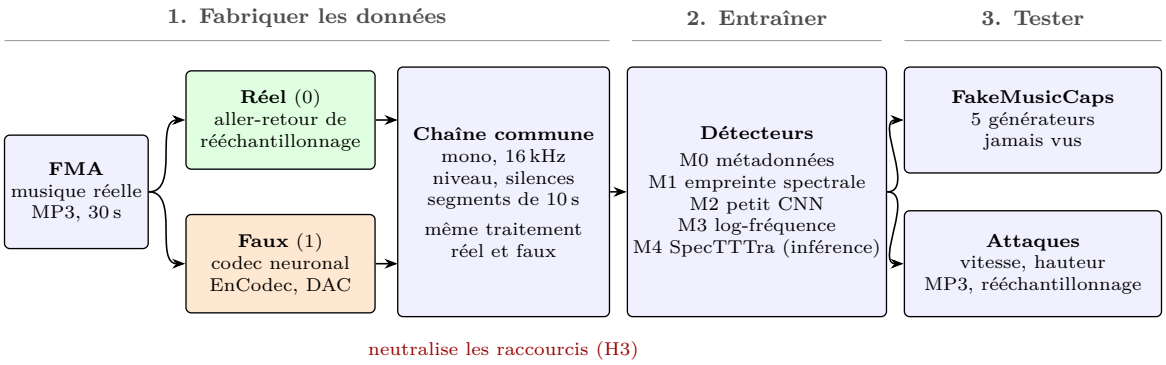


Figure 1: Le projet. Les « faux » d'entraînement viennent d'un codec, pas d'un générateur ; la chaîne commune évite que le détecteur lise le format ; le test porte sur de vrais générateurs et sur des attaques. (0) et (1) sont les étiquettes.

Les quatre hypothèses, rappelées ici car tout le cours y prépare : **H1** les artefacts viennent du *décodeur* (un détecteur transfère mieux vers les générateurs dont le décodeur ressemble à celui qui a fabriqué ses faux) ; **H2** les faux de codec se transfèrent *en partie*, surtout vers MusicGen (AUC $< 0{,}90$ pour les quatre autres) ; **H3** une part de la précision naïve vient de *raccourcis* mesurables par M0 ; **H4** la log-fréquence résiste à la vitesse mais pas forcément au MP3 ni au bruit. Ce sont des *hypothèses*, datées et figées avant l'entraînement ; une hypothèse réfutée reste un résultat valide.

2 Le signal audio

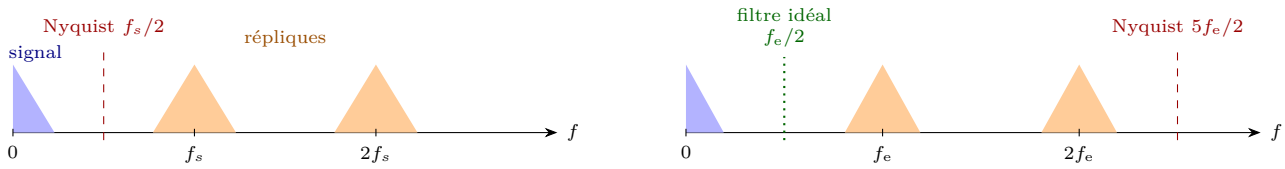
2.1 Échantillonnage, Nyquist, repliement, quantification[N1]

Un son est une fonction continue $x_c(t)$; un fichier en contient les valeurs à intervalles réguliers, $x[n] = x_c(nT_s)$, avec $f_s = 1/T_s$ la *fréquence d'échantillonnage*. Échantillonner *périodise* le spectre : $X_s(f) = f_s \sum_{k\in\mathbb{Z}} X_c(f - kf_s)$, c'est-à-dire des *répliques* du spectre d'origine centrées sur les multiples de f_s (figure 2a). Elles ne se recouvrent pas si le signal ne contient rien au-dessus de la **fréquence de Nyquist** $f_s/2$. Sinon, une composante à $f_0 > f_s/2$ se *replie* (*aliasing*) à $|f_0 - kf_s|$ pour le k le plus proche : un sinus à 13 kHz échantillonné à 22,05 kHz réapparaît à 9,05 kHz (vérifié numériquement : pic à 9 050 Hz). On évite le repliement en filtrant passe-bas *avant* de réduire la fréquence d'échantillonnage. Conséquence directe : un fichier à f_s ne contient *rien* au-dessus de $f_s/2$.

La *quantification* arrondit chaque échantillon sur b bits ; pour un sinus pleine échelle, le rapport signal sur bruit vaut environ $6{,}02\,b + 1{,}76$ dB[M], soit ≈ 98 dB à 16 bits. Un CD stéréo (44,1 kHz, 16 bits) représente $44\,100 \times 16 \times 2 = 1\,411$ kbit/s.

Lien avec le projet

FakeMusicCaps est en 16 kHz : rien au-dessus de 8 kHz. FMA est en MP3 (coupure passe-bas du codec, §2.5). Si les « réels » et les « faux » n'ont pas la même bande passante, le détecteur peut lire la *coupure* et non l'artefact : c'est un raccourci (H3). D'où la fréquence commune (16 kHz) et le rééchantillonnage



(a) signal échantillonné à f_s (b) après insertion de 4 zéros (facteur 5)

Figure 2: (a) Échantillonner replique le spectre tous les f_s. (b) Suréchantillonner par insertion de zéros (facteur 5) replique le spectre de l'entrée (cadence f_e) dans la nouvelle bande : $\lfloor 5/2 \rfloor + 1 = 3$ lobes (0, f_e, $2f_e$). Un filtre passe-bas idéal les supprimerait ; un filtre appris ne le fait qu'imparfaitement.

identique des deux classes (§3).

2.2 DFT, fenêtrage, STFT[N2]

Intuition : la DFT mesure la corrélation du signal avec des sinusoïdes complexes de fréquences kf_s/N ; son module $|X[k]|$ donne l'amplitude à cette fréquence et son argument la phase. Un sinus pur donne un pic, un clic un spectre plat. La transformée de Fourier discrète d'un bloc $x[0..N-1]$ est $X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-2\pi ikn/N}$; le bin k correspond à la fréquence kf_s/N et la résolution est $\Delta f = f_s/N$ (la FFT la calcule en $O(N \log N)$). Observer un bloc revient à multiplier le signal par une *fenêtre* $w[n]$, donc à *convoluer* son spectre par celui de la fenêtre : un sinus pur s'étale en un lobe principal et des lobes secondaires (*fuite spectrale*). Fenêtre rectangulaire : lobe principal étroit (environ 2 bins entre annulations) mais lobes secondaires élevés (premier à -13 dB) ; Hann ou Hamming : lobe principal environ deux fois plus large (4 bins), lobes secondaires bien plus bas (premier lobe à environ -31 dB pour Hann, -43 dB pour Hamming)[M]. Compléter par des zéros (*zero-padding*) interpole le spectre mais *n'ajoute aucune résolution.*

La **STFT** applique cette analyse à des fenêtres glissantes de longueur N et de pas (*hop*) H :

$$X[m,k] = \sum_{n=0}^{N-1} x[n+mH]\, w[n]\, e^{-2\pi ikn/N}.$$

Le *compromis temps-fréquence* est incontournable : une longue fenêtre résout finement les fréquences mais brouille le temps, et inversement (inégalité de Gabor[M]). À $f_s = 16$ kHz : $N = 1\,024$ donne une fenêtre de 64 ms et $\Delta f \approx 15{,}6$ Hz ; $N = 256$ donne 16 ms et 62,5 Hz. Un extrait de 10 s avec $H = 256$ produit $160\,000/256 = 625$ trames.

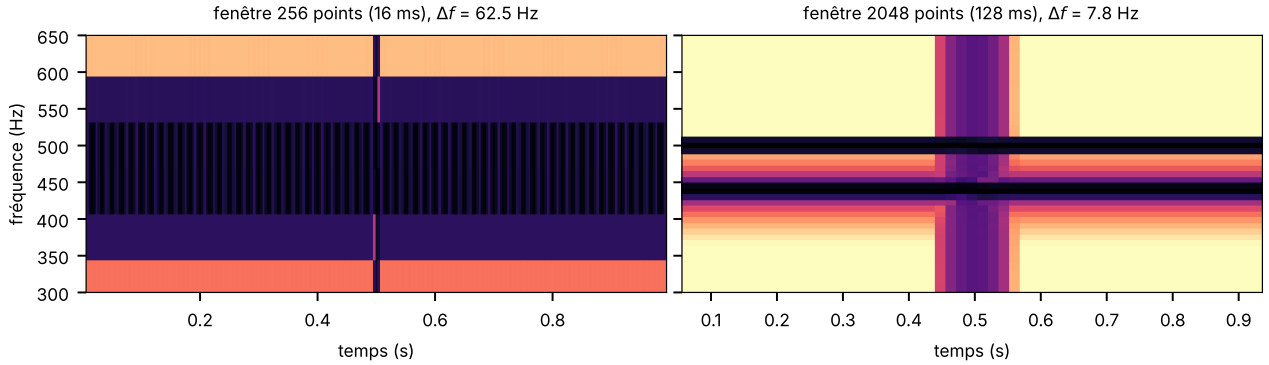


Figure 3: Deux sinus à 440 et 500 Hz et un clic à 0,5 s (foncé = énergie élevée). Fenêtre courte (à gauche) : le clic est net mais les deux sinus fusionnent en une bande. Fenêtre longue (à droite) : les sinus sont séparés mais le clic s'étale sur la durée de la fenêtre. Calcul personnel.

Lien avec le projet

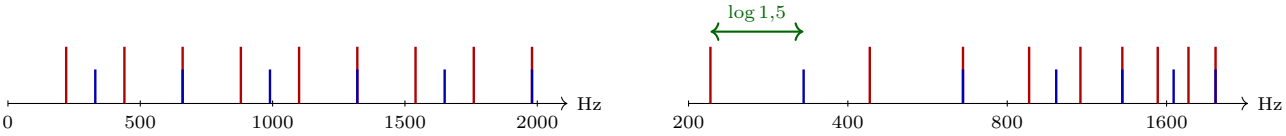
Les artefacts de codecs sont des *pics fins et stationnaires* (§5.2) : ils demandent une *longue* fenêtre (Δf bien inférieur à l'espacement des pics) et se moyennent dans le temps sans perte. À l'inverse, un spectrogramme à courte fenêtre ou mel les efface. Ce choix de représentation est une *ablation* du projet (§6).

2.3 Représentations : amplitude, dB, mel, log-fréquence, CQT[N3]

Le spectrogramme est $|X[m,k]|$ (amplitude) ou $|X|^2$ (puissance) ; on le passe en décibels, $20 \log_{10} |X|$, car l'oreille et la dynamique musicale sont logarithmiques. La phase est souvent jetée : le meilleur résultat d'Afchar et al. vient de l'*amplitude* seule[V]. Trois façons de recadrer l'axe des fréquences :

- **Mel** : $m = 2595 \log_{10}(1 + f/700)$[M], des filtres triangulaires étroits en grave et larges en aigu. Les MFCC sont la DCT du log-mel (timbre, parole). Les filtres larges moyennent *plusieurs bins* : ils effacent les pics fins.
- **Log-fréquence** : un octave est un doublement (A_2=110, A_3=220, A_4=440 Hz), un demi-ton un facteur $2^{1/12}$.
- **CQT** (*constant-Q*) : $f_k = f_{\min} 2^{k/B}$ avec B bins par octave, largeur de bin $\propto f_k$, donc $Q = f_k/\delta f_k = 1/(2^{1/B} - 1)$ constant, et fenêtre de longueur $N_k \approx Q f_s / f_k$: longue en grave, courte en aigu.

Intérêt pour un réseau : les harmoniques d'une note de fondamentale f_0 sont en $n f_0$; sur un axe *linéaire*, leur espacement vaut f_0 (le motif change avec la hauteur) ; sur un axe *logarithmique*, leurs positions sont $\log f_0 + \log n$: le motif est *identique, simplement translaté* de $\log f_0$ (figure 4). Un CNN, équivariant aux translations, en profite. Müller et al. trouvent que CQT et log-spectrogramme font environ 37 % d'EER de moins que le mel en moyenne, pour la détection de fausses voix[V] (leur jeu « in the wild » compte 37,9 h et les EER se dégradent jusqu'à environ 1 000 % hors du jeu d'entraînement ; des entrées de 4 s font moins bien que des extraits entiers).



(a) axe linéaire : espacements 220 et 330 Hz

(b) axe log : même motif, translaté

Figure 4: Harmoniques de deux notes ($f_0 = 220$ Hz en rouge, 330 Hz en bleu, rapport 3/2). En axe logarithmique, la seconde est la première translatée de $\log 1{,}5$: c'est la propriété que M3 exploite pour la vitesse (§7).

2.4 Rééchantillonnage, vitesse, hauteur, étirement[N4]

Quatre opérations qu'il faut ne *jamais* confondre :

- **Rééchantillonner** ($f_1 \to f_2$) : le contenu en Hz ne bouge pas ; on suréchantillonne (insertion de zéros, puis filtre) et/ou on décime (filtre anti-repliement, puis sous-échantillonnage). Passer à 16 kHz coupe tout au-dessus de 8 kHz.
- **Changer la vitesse** d'un facteur a : lire les mêmes échantillons à un autre rythme. Toutes les fréquences sont multipliées par a, la durée est divisée par a : $f' = af$, soit $\log f' = \log f + \log a$ (une *translation* en log-fréquence, de $12 \log_2 a$ demi-tons : $\times 0{,}7 \to -6{,}2$, $\times 1{,}1 \to +1{,}7$, $\times 1{,}4 \to +5{,}8$).
- **Étirer** le temps : changer la durée sans toucher à la hauteur (vocodeur de phase) ; le spectre reste en place en Hz.
- **Décaler la hauteur** (*pitch shift*) : étirement puis changement de vitesse pour retrouver la durée d'origine.

Effet sur un « peigne » de pics $n f_0$ (§5.2) : un rééchantillonnage ne le déplace pas ; un étirement non plus ; un changement de vitesse ou de hauteur le remplace par $n f_0'$ avec $f_0' = a f_0$, ce qui casse un détecteur calé en Hz mais est une pure translation sur l'axe log. Cela cadre avec les observations d'Afchar et al. (stable sous étirement, fragile sous décalage de hauteur)[V], *interprétation* de ma part.

2.5 Compression MP3 et ingénierie audio pratique[N5, N27]

Le MP3 est un codage *avec perte* fondé sur la perception : un modèle psychoacoustique estime quels sons sont masqués par leurs voisins (25 bandes critiques ; masquage temporel de 50 à 200 ms), une transformée en cosinus discrète modifiée (MDCT) découpe le signal en lignes spectrales, on quantifie grossièrement ce qui est masqué, puis on code sans perte (Huffman). Un CD à 1 411 kbit/s passe à 128 kbit/s, soit environ 11:1[R]. Détails de bas niveau (banc de 32 sous-bandes, 576 lignes spectrales par granule)[M].

Ce qui compte pour le projet est la **coupure passe-bas** : l'encodeur supprime les hautes fréquences à bas débit, et FMA entier est en MP3. J'ai *mesuré* cette coupure (ffmpeg 6.1.1, libmp3lame, bruit blanc stéréo à 44,1 kHz, fréquence où le spectre moyen passe 30 dB sous son niveau médian) :

Débit (kbit/s)	64	96	128	192	256	320
Coupure mesurée (kHz)	11,0	15,2	16,7	18,8	19,5	20,2

En mono, la même mesure à 128 kbit/s donne environ 20 kHz : la coupure dépend du mode, du débit *et* de l'encodeur (mesurer, ne pas supposer ; lab L2). Sur un signal musical réel d'autres effets s'ajoutent. Enfin, ffmpeg compense le retard de l'encodeur grâce à l'en-tête du fichier (mesuré : 0 échantillon de décalage) ; ce n'est pas vrai de tous les décodeurs.

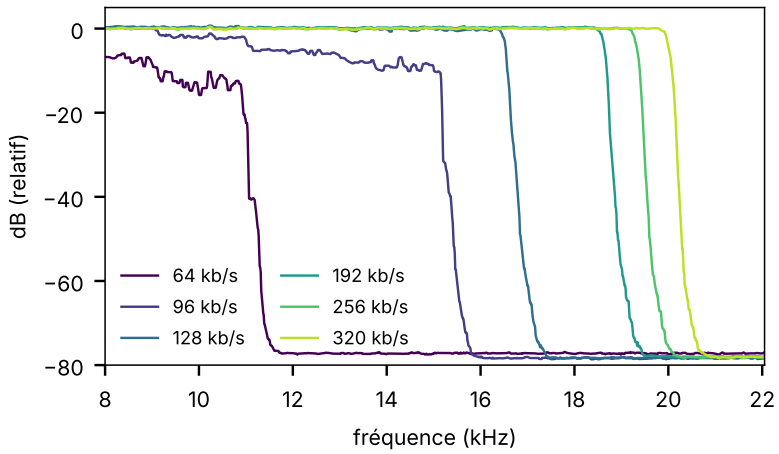


Figure 5: Spectre moyen d'un bruit blanc stéréo à 44,1 kHz après passage MP3 (ffmpeg 6.1.1, libmp3lame), aux six débits du tableau. Mesure personnelle (lab L2).

Ingénierie. Quelques commandes suffisent à construire la chaîne du §3 :

```
ffmpeg -i in.mp3 -ac 1 -ar 16000 -af aresample=resampler=soxr out.wav   # mono, 16 kHz
ffmpeg -i in.wav -c:a libmp3lame -b:a 128k attaque.mp3                  # attaque MP3
ffmpeg -i in.wav -af loudnorm=I=-23 norm.wav                            # normalisation LUFS (EBU R128)
```

Niveau : RMS en dBFS $= 20\log_{10}\sqrt{\overline{x^2}}$ (la sonie en LUFS pondère les fréquences). Toujours conserver le même filtre de rééchantillonnage pour les deux classes.

3 Chaîne commune, raccourcis et contrôles négatifs

3.1 Le raccourci : définition et précédents[N19]

Geirhos et al. (arXiv 2004.07780)[V] appellent *shortcut learning* le cas où un modèle résout une tâche par une règle de décision qui réussit sur le test i.i.d. mais échoue dès que les conditions changent (test hors distribution). Ils distinguent, parmi toutes les règles possibles, celles qui réussissent à l'entraînement, puis celles qui réussissent en i.i.d. (dont les *raccourcis*), enfin la *solution voulue*, qui généralise aussi hors distribution. Exemples : une vache reconnue par l'herbe, un classifieur de pneumonie qui lit la marque de l'hôpital, BERT qui détecte le mot « not ». Trois origines : les *données* (biais systématiques), la *règle de décision* (l'apprentissage discriminatif retient n'importe quel indice suffisant) et le *biais inductif*. Trois conseils de diagnostic : n'attribuer une capacité de haut niveau que si aucun raccourci ne l'explique (« canon de Morgan »), comparer à des baselines fortes qui *n'ont pas accès* à la caractéristique voulue, remplacer les tests i.i.d. par des tests hors distribution.

Trois précédents audio, directement transposables :

- **Silence** (Müller et al., arXiv 2106.12914)[V] : dans ASVspoof 2019, les enregistrements authentiques ont des silences initiaux et finaux plus longs. Un réseau dense qui ne voit *que* la durée du silence initial atteint un EER de 15,12 % (hasard : 50 %) ; rogner le silence fait passer RawNet2 de 3,61 % à 15,50 % d'EER ; ajouter la durée du silence comme caractéristique n'aide pas (les modèles l'extraient déjà). Leçon des auteurs, que je retiens : le silence n'est pas illégitime en soi, c'est sa répartition inégale qui l'est ; il faut l'équilibrer ou le supprimer.
- **Format** (Oh, ArtifactNet)[V] : avant un entraînement conscient des codecs, le modèle classait comme IA 98,7 % des pistes FMA en MP3 ; après, 8,0 % sur un sous-ensemble de 1 350 pistes MP3 « difficiles ».
- **Généralisation** (Müller et al. 2022)[V] : hors du jeu d'entraînement, les EER se dégradent jusqu'à environ 1 000 %.

3.2 Inventaire des raccourcis possibles ici

Indice	Pourquoi il fuit	Neutralisation / mesure
Bande passante, fréquence native	FMA est en MP3 (coupure passe-bas) ; les générateurs ont des fréquences natives différentes (16, 32, 44,1 kHz)	Rééchantillonnage commun ; caractéristique « bande passante » dans M0
Format et codec amont	MP3 contre WAV ; l'artefact d'un MP3 se confond avec celui d'un codec	Même chaîne pour les deux classes ; M0 ; attaque MP3 *explicite*
Durée, silences, début/fin	Génération par fenêtres de 10 s ; silences de début de piste	Segments de 10 s à position aléatoire ; silence dans M0 ; modèle « silence seul »
Niveau sonore	Normalisation différente selon la source ; écrêtage	Normalisation commune (RMS ou LUFS) ; niveau dans M0
Mono/stéréo, bits, étiquettes (ID3)	Métadonnées de fichier	Tout convertir (mono, 16 kHz, flottant) ; jeter les étiquettes
Contenu (artiste, genre, voix)	Deux extraits d'un même artiste se ressemblent	Découpage *par artiste* (et par légende pour les générateurs)

3.3 La chaîne commune

Principe (figure 1) : les deux classes subissent *exactement les mêmes* opérations, sauf celle que l'on veut détecter. Pour un morceau FMA x et un codec de fréquence native f_c (24 kHz pour EnCodec, 44,1 kHz pour DAC) :

réel : $x \xrightarrow{\text{mono}} \xrightarrow{\to f_c} \xrightarrow{\to 16\,\text{kHz}}$ niveau $\to$ segments faux : $x \xrightarrow{\text{mono}} \xrightarrow{\to f_c} \boxed{\text{codec}} \xrightarrow{\to 16\,\text{kHz}}$ niveau $\to$ segments

avec le même rééchantillonneur (`soxr`), le même niveau cible et les mêmes segments de 10 s. Conséquences à connaître : (i) le MP3 d'origine est présent dans les deux classes, il ne discrimine pas ; (ii) la fréquence d'ancrage de 16 kHz borne l'analyse à 8 kHz, donc, pour EnCodec 24 kHz (un pic tous les 75 Hz, §5.2), à environ 106 pics au lieu de 160 ; (iii) on ne peut rien conclure sur les artefacts au-dessus de 8 kHz (les papiers de Deezer travaillent en 5–16 kHz[V]).

3.4 Détecteur de métadonnées M0 et contrôles[N20]

M0 est un classifieur qui ne voit *aucun signal audio*, seulement des descripteurs du fichier : durée, fréquence native, bande passante estimée (plus haute fréquence dont le niveau dépasse −60 dB), niveau RMS, proportion d'échantillons nuls, durée de silence initial, écrêtage. On entraîne (régression logistique ou arbres) sur les mêmes découpages et l'on rapporte l'AUC avec un intervalle de confiance bootstrap groupé (§4.4). *(hypothèse du projet : si la chaîne est saine, M0 est au hasard)* ; un intervalle qui exclut 0,5 prouve un raccourci, qu'il faut corriger *avant* d'interpréter les autres détecteurs (précisément la démarche de Müller et al.). On parle de *M0 résiduel* quand on le mesure *après* la chaîne commune.

Contrôles à prévoir :

- **négatifs** (doivent donner AUC ≈ 0,5) : M0 ; étiquettes permutées ; modèle « silence seul » ;
- **positif** (doit donner AUC ≈ 1) : un détecteur entraîné *et testé* sur le même codec, preuve que l'instrument voit l'artefact quand il existe.

Une expérience sans contrôle ne prouve rien : si le positif échoue, le montage est défaillant ; si le négatif dépasse 0,5, il y a une fuite.

4 Apprentissage et évaluation

4.1 CNN sur spectrogramme[N6]

Un spectrogramme se traite comme une image (fréquence × temps), mais *les deux axes ne sont pas équivalents* (slides de G. Richard, MVA[V]) : décaler un événement dans le temps le laisse inchangé, alors que décaler un spectre de Δf sur un axe *linéaire* ne revient pas à changer la hauteur (les harmoniques s'espacent autrement, §2.3). Sur un axe logarithmique, si. Trois notions suffisent :

- **Convolution et équivariance** : un filtre appris glisse sur l'image ; translater l'entrée translate la carte de sortie.
- **Pooling** : le max-pooling rend la sortie (approximativement) *invariante* à de petits décalages, au prix de la résolution. Pour un détecteur de pics fins, pooler trop tôt l'axe des fréquences peut détruire l'indice : à ablater.

- **Champ réceptif** : avec $r_0 = 1$, $j_0 = 1$ (saut), une couche de noyau k et de pas s donne $r_l = r_{l-1} + (k-1)\, j_{l-1}$ et $j_l = j_{l-1}\, s$. Trois convolutions 3×3 sans pas : $r = 7$. Avec un max-pooling 2×2 (pas 2) après chaque convolution : $r = 3 \to 4 \to 8 \to 10 \to 18 \to 22$; on voit 22×22 pixels.

Les convolutions dilatées et les pas élargissent le champ sans multiplier les paramètres (MVA, §7). Le CNN d'Afchar et al.[V] a six couches convolutives, environ 1,6 M de paramètres, et lit un spectrogramme d'amplitude sur des extraits de 0,8 s. Pour le choix des entrées et des couches, le tutoriel de Choi et al. (arXiv 1709.04396)[V] fait référence (représentations d'entrée, convolutions 1D et 2D, pooling, conseils de conception) ; il ne traite *pas* des fuites entre artistes (§4.4).

4.2 Transformers sur spectrogramme[N7]

L'*Audio Spectrogram Transformer* (AST, arXiv 2104.01778)[R] découpe le spectrogramme en patchs, les projette en jetons et applique de l'attention : c'est le premier classifieur audio sans convolution (AudioSet 0,485 mAP, ESC-50 95,6 %, Speech Commands V2 98,1 %)[R]; log-mel à 128 bandes, patchs 16×16 avec recouvrement et initialisation depuis un ViT/DeiT d'ImageNet[M]. Le nombre de jetons d'un découpage en patchs 2D croît comme le *produit* (temps)×(fréquence). **SpecTTTra** (SONICS, arXiv 2408.14080)[V] tokenise séparément le temps et la fréquence par des convolutions 1D : la croissance devient *additive*, ce qui permet des extraits longs (F1 de 0,97 à 120 s ; au contraire, à 5 s c'est ConvNeXt qui est le meilleur, 0,90). Le projet n'utilise SpecTTTra qu'en *inférence* (M4, point de comparaison externe).

4.3 Métriques de détection et calibration[N8]

Un détecteur produit un score $s(x)$ (« faux » si $s > \tau$). Les taux de vrais et de faux positifs sont $\mathrm{TPR}(\tau) = P(s > \tau \mid y{=}1)$ et $\mathrm{FPR}(\tau) = P(s > \tau \mid y{=}0)$; la courbe ROC les trace en balayant τ.

- **AUC** : aire sous la ROC, égale à $P\big(s(X^+) > s(X^-)\big)$ (plus $\frac{1}{2}$ pour les égalités) : la probabilité qu'un faux tiré au hasard ait un score supérieur à celui d'un réel, soit $\mathrm{AUC} = \frac{1}{n^+ n^-} \sum_i \sum_j \mathbf{1}[s_i^+ > s_j^-]$ (statistique de Mann–Whitney). AUC = 0,5 signifie « hasard ». Elle ne dépend que du *rang* des scores.
- **EER** : valeur commune de FPR et de $1 - \mathrm{TPR}$; TPR@1 %FPR : taux de détection quand on tolère 1 % de faux positifs (point de fonctionnement réaliste).

Exemple à six scores : faux {0,9; 0,8; 0,4}, réels {0,7; 0,3; 0,2}. Sur les 9 paires (faux, réel), 8 sont bien ordonnées (seule $0{,}4 < 0{,}7$ échoue) : $\mathrm{AUC} = 8/9 \approx 0{,}89$. Pour $\tau = 0{,}5$: $\mathrm{TPR} = 2/3$, $\mathrm{FPR} = 1/3$, donc $\mathrm{EER} = 1/3$ (vérifié avec scikit-learn).

Pourquoi l'AUC ne suffit pas. Elle est invariante à toute transformation monotone des scores, mais l'exactitude et le F1 dépendent du seuil τ figé sur le jeu d'entraînement. Si la distribution des scores se décale (autre générateur, attaque), l'AUC peut rester haute et le F1 s'effondrer : Dugelay et al. rapportent, sous décalage de hauteur, AUC 0,963 mais F1 0,279[V]; Park et al. insistent pour évaluer aussi le point de fonctionnement[V]. On rapporte donc l'AUC *et* TPR@1 %FPR avec un seuil figé sur des données propres.

Calibration (Guo et al., arXiv 1706.04599)[V]. Un classifieur est calibré si $P(\hat{Y} = Y \mid \hat{P} = p) = p$. On la mesure par le *diagramme de fiabilité* (exactitude par intervalle de confiance) et par l'*ECE* : $\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \big|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\big|$ (M intervalles ; MCE : le maximum au lieu de la somme) ; on suit aussi la NLL. Le *temperature scaling* divise les logits par un scalaire $T > 0$ ajusté en minimisant la NLL sur un jeu de validation : il ne change ni l'argmax ni l'ordre des scores, donc ni l'exactitude ni l'AUC. Les auteurs notent que la normalisation par lots, une faible régularisation et une grande capacité dégradent la calibration. Le score de Brier $\overline{(p-y)^2}$ est un complément usuel[M]. Afchar et al. observent des sorties presque toujours proches de 0 ou 1, avec la classe « réel » comme défaut sous perturbation[V]: la calibration y est un vrai sujet.

4.4 Incertitude, comparaisons, découpages[N9, N26]

Découper par groupe. Deux extraits d'un même morceau, ou de deux morceaux d'un même artiste, partagent timbre, mastering, voix : un découpage aléatoire laisse fuiter l'information entre entraînement et test et gonfle les scores. On découpe donc *par artiste* pour FMA (identifiant d'artiste dans `tracks.csv`) et *par légende* pour FakeMusicCaps (une légende produit un extrait par générateur). Chaque extrait de 10 s est une *unité corrélée* à ses voisins.

Bootstrap groupé. Pour un intervalle de confiance sur l'AUC, on tire les *groupes* (artistes, légendes) avec remise, on recalcule l'AUC, on répète ($B = 1\,000$ à $10\,000$) et l'on prend les centiles 2,5 % et 97,5 %. Tirer les extraits individuellement sous-estimerait l'incertitude. **Test apparié** : pour comparer deux détecteurs A et B, on tire les *mêmes* groupes pour les deux et l'on calcule $\Delta\mathrm{AUC} = \mathrm{AUC}_A - \mathrm{AUC}_B$; un intervalle qui exclut 0 est la convention « A fait mieux » du projet (alternative : le test de DeLong). Avec m comparaisons, la

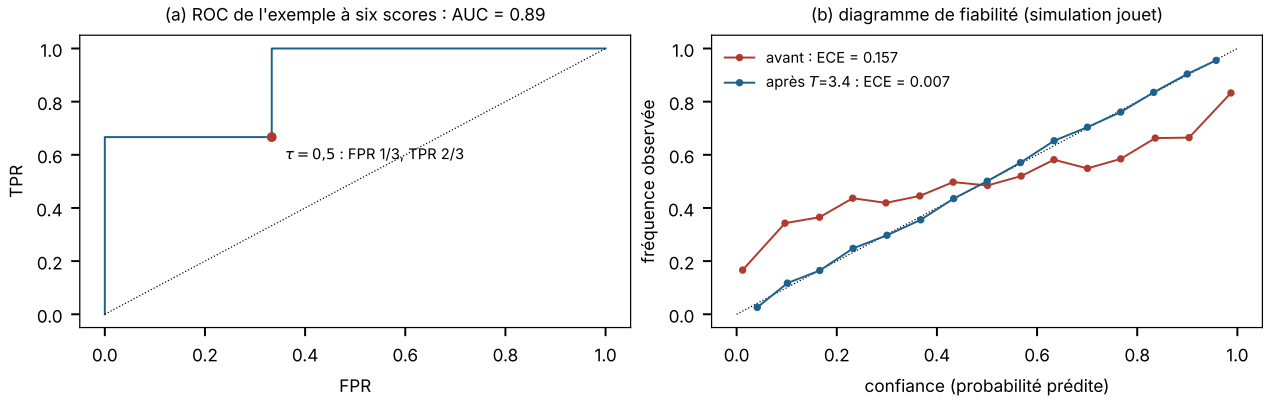


Figure 6: (a) ROC de l'exemple à six scores (AUC = 8/9). (b) Simulation jouet de 20 000 exemples à logits sur-confiants : le *temperature scaling* ($T = 3{,}4$, par minimisation de la NLL ; ajusté ici sur les mêmes données par simplicité, en pratique sur une validation) ramène l'ECE de 0,157 à 0,007 sans toucher à l'AUC (0,827 avant et après). Calculs personnels.

correction de Holm classe les p-valeurs $p_{(1)} \le \cdots \le p_{(m)}$ et rejette $H_{(i)}$ tant que $p_{(i)} \le \alpha/(m-i+1)$.

```
def boot_delta(y, sA, sB, g, B=2000, seed=0):            # y: etiquettes, sA/sB: scores, g: groupe
    rng = np.random.default_rng(seed); G = np.unique(g)
    idx = {k: np.flatnonzero(g == k) for k in G}; d = []
    for _ in range(B):
        i = np.concatenate([idx[k] for k in rng.choice(G, len(G), replace=True)])
        if len(np.unique(y[i])) == 2:                     # memes groupes pour A et B
            d.append(roc_auc_score(y[i], sA[i]) - roc_auc_score(y[i], sB[i]))
    return np.percentile(d, [2.5, 97.5])
```

Trois variabilités distinctes, à ne pas confondre : le *bootstrap* mesure la variabilité due au *jeu de test fini* ; la *variance entre graines* celle de l'*entraînement* ; la *variabilité des données* celle du choix du jeu d'entraînement. Aucune ne remplace les autres (lab L7).

Lien avec le projet

Conventions du projet : « au-dessus du hasard » = l'IC à 95 % de l'AUC (bootstrap groupé) exclut 0,5 ; « mieux » = l'IC à 95 % apparié de ΔAUC exclut 0 ; « transfert fort » = AUC $\ge$ 0,90 (seuil indicatif, à valider avec l'encadrant).

5 Génération audio neuronale : d'où viennent les artefacts

5.1 Auto-encodeurs, VAE, GAN, diffusion[N10, N11, N16]

Un **auto-encodeur** compresse x en $z = E(x)$ puis le reconstruit, $\hat{x} = D(z)$, en minimisant $\|x - \hat{x}\|^2$; le goulot d'étranglement force à ne garder que l'essentiel. Un **VAE** fait produire à l'encodeur une moyenne μ et un écart-type σ, tire $z = \mu + \sigma \odot \varepsilon$ ($\varepsilon \sim \mathcal{N}(0, I)$, *reparamétrisation*) et maximise la borne $\mathbb{E}_q[\log p(x \mid z)] - \mathrm{KL}\big(q(z \mid x) \,\|\, p(z)\big)$ (Weng, VAE[V]). Un **GAN** oppose un générateur G à un discriminateur D : $\min_G \max_D \ \mathbb{E}[\log D(x)] + \mathbb{E}[\log(1 - D(G(z)))]$. Les codecs et les vocodeurs (HiFi-GAN) utilisent des variantes : perte *hinge* ($L_D = \mathbb{E}[\max(0, 1 - D(x))] + \mathbb{E}[\max(0, 1 + D(\hat{x}))]$, $L_G = -\mathbb{E}[D(\hat{x})]$, forme usuelle[M]) sur plusieurs discriminateurs (formes d'onde à plusieurs échelles, STFT), plus une perte de *feature matching* (distance entre les activations du discriminateur sur x et $\hat{x}$) et une perte spectrale multi-échelle sur mel. Les **modèles de diffusion** (Weng, diffusion[V]) bruitent x_0 par $q(x_t \mid x_{t-1}) = \mathcal{N}(\sqrt{1-\beta_t}\, x_{t-1}, \beta_t I)$, ce qui donne en forme close $x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon$ avec $\bar{\alpha}_t = \prod_{s \le t}(1-\beta_s)$; un réseau ε_θ apprend à prédire le bruit, $L = \mathbb{E}\|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$, puis on inverse le processus. La *diffusion latente* fait cela dans l'espace z d'un VAE, conditionnée par du texte via de l'attention croisée, avec le *guidage sans classifieur* $\tilde{\varepsilon} = (w+1)\varepsilon_\theta(x_t, c) - w\,\varepsilon_\theta(x_t)$.

Idée directrice : quelle que soit la machinerie en amont (LM, diffusion, quantification), la *dernière étape* — décodeur de codec, décodeur de VAE, vocodeur — transforme une représentation à basse cadence en forme d'onde par des couches de *suréchantillonnage*. C'est là que naît l'empreinte.

5.2 Suréchantillonnage et origine des pics spectraux[N12, N25]

Convolution transposée. Avec un pas s et un noyau de taille K, elle insère $s-1$ zéros entre les échantillons d'entrée, puis convolue avec le noyau appris : $y[n] = \sum_i x[i]\, h[n - si]$, de longueur $(L-1)s + K$. Si K n'est pas multiple de s, le recouvrement est *inégal* : avec $K = 3$, $s = 2$, les sorties reçoivent alternativement 2 et 1

contributions ; avec $K = 5$, $s = 3$, le motif est $1, 2, 2, 1, 2, 2, \ldots$ (vérifié par un petit calcul). Distill (Odena et al.)[V] montre que ces motifs en damier apparaissent même à l'initialisation aléatoire ; le remède est de redimensionner (plus proche voisin) *puis* de convoluer. Pons et al. (arXiv 2010.14356)[R] distinguent des artefacts *tonaux* (dus à l'insertion de zéros) et de *filtrage*, et retiennent le plus proche voisin comme meilleur compromis ; les notebooks de Dolby (licence MIT)[V] permettent de les reproduire (conv. transposée, plus proche voisin, interpolation linéaire, sous-pixel).

Explication de Fourier. Le théorème de convolution donne $\mathcal{F}[s * r] = \mathcal{F}[s] \cdot \mathcal{F}[r]$. Insérer des zéros avec un facteur k donne $u[n] = s[n/k]$ si $k \mid n$, sinon 0, donc $U(e^{i\omega}) = \sum_m s[m] e^{-i\omega k m} = S(e^{ik\omega})$: le spectre est comprimé d'un facteur k et *répliqué* k fois (figure 2b) ; le filtre r appris qui suit n'élimine ces images qu'imparfaitement. Afchar et al. (ISMIR 2025, arXiv 2506.19108)[R] en tirent que, si f_e est la cadence *avant* suréchantillonnage, la sortie présente des pics aux fréquences $n f_e$, $n \in \{0, \ldots, \lfloor k/2 \rfloor\}$; avec L couches, $P_{\max} = \prod_i k_i$ et $\lfloor P_{\max}/2 \rfloor + 1$ pics. Pour EnCodec, les pas $\{8, 5, 4, 2\}$ donnent $P_{\max} = 320$ et **161 pics**, un tous les 75 Hz jusqu'à 12 kHz. Intuition temporelle : un motif qui se répète toutes les 320 échantillons (13,3 ms à 24 kHz) a pour spectre des raies aux multiples de $24\,000/320 = 75$ Hz. Ils dépendent de l'*architecture* (les pas), non des poids ni des données (vérifié sur DAC par plusieurs graines et jeux de données) ; leur *amplitude*, elle, dépend des filtres.

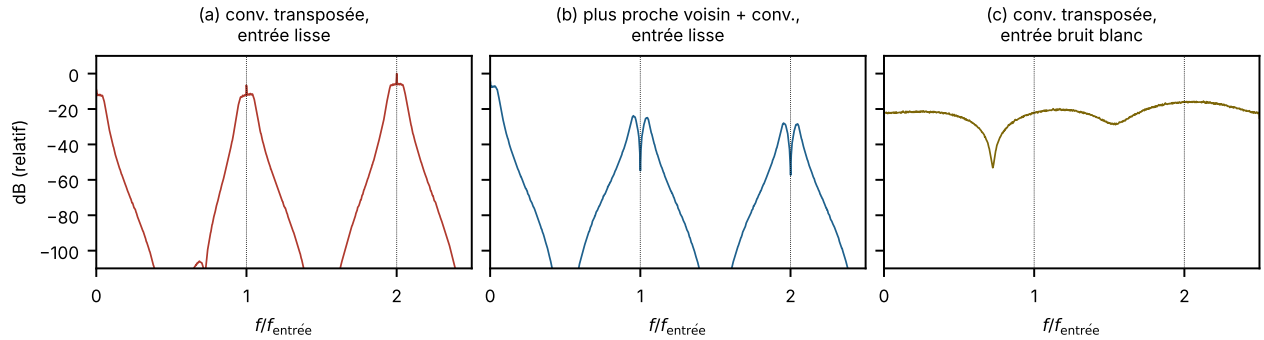


Figure 7: Spectre moyen (400 réalisations) d'un suréchantillonneur jouet : facteur 5, noyau 7, poids aléatoires. (a) Conv. transposée, entrée *lisse* (basses fréquences plus composante continue) : lobes à 0, f_e, $2f_e$ d'amplitude comparable. (b) Plus proche voisin puis conv. : lobes en f_e et $2f_e$ creusés, environ 20 dB plus bas dans ce tirage. (c) Conv. transposée, entrée en *bruit blanc* : spectre moyen lisse, sans pics. Mesure personnelle (lab L4).

La figure 7c corrige un piège : avec un bruit blanc en entrée, la périodicité du suréchantillonnage se voit dans la *variance temporelle* (période s) mais pas dans le spectre de puissance moyen. Les pics exigent une entrée *structurée en basses fréquences*, comme les représentations latentes lisses d'un codec ou une composante continue (biais).

Décodeur	f_e	$P_{\max}$	**Pics**	**Remarque**
EnCodec 24 kHz (strides 8, 5, 4, 2)	75 Hz	320	161	jusqu'à 12 kHz
EnCodec 32 kHz (MusicGen)	50 Hz	640	321	jusqu'à 16 kHz
DAC 44,1 kHz (86 Hz)	86 Hz	≈ 512	≈ 257	jusqu'à 22 kHz
VAE de Stable Audio Open (21,5 Hz)	21,5 Hz	$\approx 2\,048$	$\approx 1\,025$	un pic tous les 21,5 Hz

Calculs personnels d'après les cadences annoncées ($P_{\max} = f_{\text{audio}}/f_e$), valables si tout le suréchantillonnage se fait par insertion de zéros ; un décodeur à plus proche voisin ou à Snake se comporte autrement. À 16 kHz (ancrage du projet), il ne reste que les pics sous 8 kHz.

5.3 Quantification vectorielle et codecs neuronaux[N13, N14]

VQ. Un dictionnaire (*codebook*) de K vecteurs c_j remplace z par son plus proche voisin, $q(z) = c_{j^\star}$, $j^\star = \arg\min_j \|z - c_j\|$; on l'entraîne (VQ-VAE) par *straight-through* (copie du gradient) avec $L = \|x - D(e)\|^2 + \|\mathrm{sg}[z_e] - e\|^2 + \beta\|z_e - \mathrm{sg}[e]\|^2$ (reconstruction, dictionnaire, engagement ; sg : arrêt de gradient), ou par moyenne mobile exponentielle du dictionnaire (Weng[V]). **RVQ** : plusieurs quantificateurs en cascade, chacun quantifiant le *résidu* du précédent : $r_0 = z$, $k_q = \arg\min_j \|r_{q-1} - c_j^q\|$, $r_q = r_{q-1} - c_{k_q}^q$, $\hat{z} = \sum_q c_{k_q}^q$ (AssemblyAI[V]). Le débit est $R = f_{\text{trame}} \times N_q \times \log_2 K$: plus de codebooks, plus de débit, meilleure reconstruction.

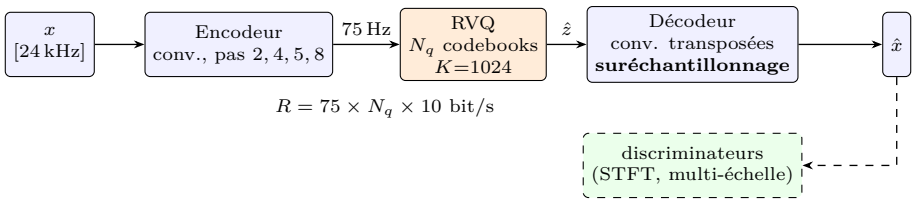


Figure 8: Codec neuronal de type EnCodec. $N_q = 8$ donne $75 \times 8 \times 10 = 6$ kbit/s. L'artefact vient du décodeur (zone grasse), pas de la quantification.

Codec	f_s	Trames	$N_q \times K$	Débit	Particularités
SoundStream[V]	24 kHz	75 Hz	8×1024 (ex.)	3–18 kbit/s	RVQ ; *quantizer dropout* (nb. de codebooks tiré au hasard) ; discr. STFT + 3 formes d'onde ; λ_{adv}=1, λ_{feat}=100, λ_{rec}=1
EnCodec[V]	24 (mono), 48 kHz (stéréo)	75 / 150 Hz	jusqu'à 32×1024	1,5–24 ; 6–24 kbit/s	LSTM ; reconstruction L1 + mel multi-échelle ; MS-STFT adversarial (hinge) + feature matching ; *loss balancer* ; LM optionnel (−40 %)
DAC[V]	44,1 kHz (aussi 24, 16)	86 Hz	9×1024	≈ 8 kbit/s (≈ 90×)	activation *Snake* ; codebooks de 8 dim. normalisés L_2 ; dropout $p = 0{,}5$; discr. STFT complexe multi-échelle et multi-bande
EnCodec de MusicGen[V]	32 kHz mono	50 Hz	4×2048	≈ 2,2 kbit/s	jetons pour un modèle de langage
Mimi[V]	24 kHz	12,5 Hz	8×2048	≈ 1,1 kbit/s	*split RVQ* : 1 niveau sémantique (distillation WavLM) + 7 acoustiques ; Transformers causaux ; données d'entraînement non confirmées

Codec-SUPERB (arXiv 2402.13071)[V] compare 6 codecs (19 modèles) sur 20 jeux de données : DAC offre le meilleur compromis débit/qualité, AcademiCodec reste bon à 2–3 kbit/s, EnCodec est une référence solide. **Musika** (arXiv 2208.08706)[V] n'est pas un RVQ : deux auto-encodeurs convolutifs empilés sur le spectrogramme STFT (magnitude et phase, iSTFT), un GAN générant dans l'espace latent, 22,05 kHz.

Lien avec le projet

Un codec n'est pas un générateur : il reçoit de la vraie musique et la « reconstruit » avec les défauts de son décodeur. Les faux de substitution du projet sont donc de la musique réelle *vue à travers un décodeur. (hypothèse du projet : H1 : les artefacts viennent du décodeur ; un détecteur transfère mieux vers un générateur dont le décodeur ressemble à celui qui a fabriqué ses faux).* Dans la matrice détecteur × générateur, EnCodec (MusicGen) est le cas favorable.

5.4 Modèles de musique et fiche des cinq générateurs[N15, N16]

MusicGen (arXiv 2306.05284)[V] est un modèle de langage Transformer *autorégressif* sur les jetons EnCodec (32 kHz, 4 codebooks, 50 Hz). Plutôt que d'aplatir les 4 flux (séquence 4× plus longue), un *motif d'entrelacement à retard* décale le codebook k de $k-1$ pas : un seul passage prédit les 4 niveaux en parallèle. Tailles 300 M, 1,5 Md et 3,3 Md de paramètres, environ 20 000 h de musique, conditionnement texte (T5) et mélodie (chroma). Les quatre autres générateurs de FakeMusicCaps sont à **diffusion latente** (§5.1) ; leur décodeur final est un VAE, parfois suivi d'un vocodeur.

Générateur	Architecture	Décodeur final	Natif	Données
MusicGen (medium)[V]	LM sur jetons EnCodec ; T5	décodeur EnCodec	32 kHz mono	≈ 20 k h
MusicLDM[V]	CLAP + diffusion latente (UNet) ; mel à 128 bandes ; mélange synchronisé sur le rythme	VAE + HiFi-GAN réentraîné sur la musique	16 kHz, 10,24 s	—
AudioLDM 2 (musique)[V]	AudioMAE + GPT-2 + diffusion latente	VAE + HiFi-GAN (variante non précisée)	16 kHz	dont FMA (106 574 pistes) et un sous-ensemble de MSD
Stable Audio Open[V]	VAE (≈ 156 M, Snake, 64 canaux latents, 21,5 Hz) + DiT (≈ 1 Md) ; T5	décodeur du VAE (sans tanh)	44,1 kHz stéréo	486 492 enregistrements (≈ 7 300 h), Freesound + FMA (licences CC)
Mustango[V]	diffusion latente de type Tango, VAE d'AudioLDM, FLAN-T5, MuNet (rythme, accords)	non précisé dans la page lue	16 kHz, 10 s max.	—

FakeMusicCaps fournit tout en **mono 16 kHz** : MusicGen et Stable Audio Open sont donc *rééchantillonnés et mélangés en mono* par les auteurs du jeu, ce qui retire ce qui est au-dessus de 8 kHz. Deux observations pour l'interprétation : les données d'entraînement de Stable Audio Open et d'AudioLDM 2 contiennent FMA, c'est-à-dire potentiellement les « réels » du détecteur ; et le **décodeur oracle** du projet (passer de la vraie musique par l'encodeur–décodeur d'un générateur ouvert, sans la diffusion) fabrique des faux au plus près de ce générateur.

6 Les détecteurs M0 à M4

	Entrée et modèle	Famille	Ce qu'il teste dans le projet
M0	descripteurs de fichier ; régression logistique ou arbres	contrôle négatif	raccourcis (H3)
M1	empreinte spectrale moyenne ; régression logistique ($\approx 10^4$ paramètres)	artefact explicite	explication de Fourier (H1)
M2	spectrogramme d'amplitude ; petit CNN ($\approx 1{,}6$ M de paramètres, à la Afchar)	appris	transfert hors famille (H1, H2)
M3	spectre moyen en log-fréquence ; corrélation croisée, max-pooling	invariant à l'échelle	vitesse, hauteur, MP3 (H4)
M4	SpecTTTra de SONICS, en inférence	référence externe	point de comparaison

6.1 Empreintes spectrales (*fakeprints*) : M1[N17]

Recette de Afchar et al. (arXiv 2506.19108)[R] : (1) calculer l'amplitude de la STFT sur de nombreux segments et la **moyenner dans le temps**, ce qui fait ressortir les pics stationnaires et écrase la musique ; (2) **soustraire** un minimum glissant (enveloppe inférieure locale) pour retirer la forme lente du spectre ; (3) **restreindre** à une bande (5–16 kHz dans l'article ; 1–8 kHz pour SONICS à 16 kHz) ; (4) entraîner une **régression logistique linéaire** (environ 10^4 paramètres : un poids par bin). Résultats annoncés : environ 99,7 à 100 % sur DAC, EnCodec et Musika ; Suno v2/v3/v3.5 et Udio 130 $\approx$ 100 % ; mais Udio 32, non vu à l'entraînement, seulement 39,8 %. Limites reconnues : réseaux séquentiels simples étudiés, interpolation non entièrement démontrée, robustesse au rééchantillonnage et à la hauteur non testée.

```python
def fakeprint(x, sr=16000, n_fft=4096, band=(1000, 8000), w=101):    # schema pedagogique
    f, _, Z = sg.stft(x, sr, nperseg=n_fft, noverlap=n_fft // 2)
    m = np.log(np.abs(Z).mean(axis=1) + 1e-9)            # 1. spectre moyen (log)
    m = m - nd.minimum_filter1d(m, size=w)               # 2. retire l'enveloppe lente
    return m[(f >= band[0]) & (f <= band[1])]            # 3. bande ; 4. -> LogisticRegression
```

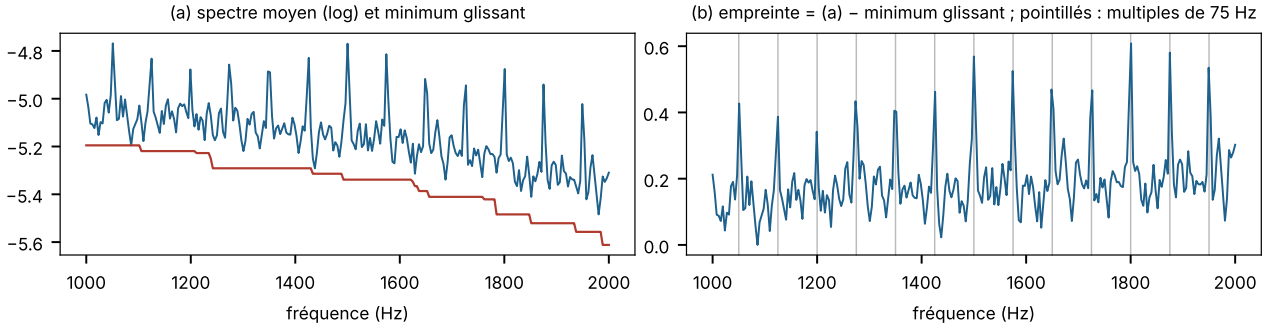


Figure 9: Le croquis ci-dessus sur un signal synthétique (10 s à 16 kHz : bruit coloré plus un peigne à 75 Hz). (a) Le spectre moyen a une pente et un plancher ; le minimum glissant (rouge) suit le plancher. (b) Une fois la pente retirée, les pics tombent sur les multiples de 75 Hz. Calcul personnel.

Ce croquis est le mien, *pas* le code de Deezer ; vérifié sur un bruit auquel on ajoute un peigne à 75 Hz (les pics ressortent, niveau 0,39 contre 0,17 entre les pics ; sans peigne, 0,17 et 0,18). Le dépôt `deezer/ismir25-ai-music-detector` fournit `compute_fakeprints.py`, `train_test_regressor.py` et un notebook de visualisation des latents EnCodec ; il demande FMA (avec le découpage du détecteur) et SONICS, est sous CC BY-NC 4.0, fait l'objet de deux demandes de brevet (déc. 2024) et n'est *pas* le détecteur de production. Choisir la fenêtre : $\Delta f = f_s/N$ doit être bien inférieur à l'espacement des pics (4 096 points à 16 kHz : 3,9 Hz pour des pics tous les 75 Hz) ; la taille w du minimum glissant doit couvrir plusieurs pics. Exemple de dimension : 4 096 points à 16 kHz donnent des bins de 3,906 Hz ; la bande 1–8 kHz contient donc 1 793 bins, soit autant de poids (plus un biais) pour la régression de M1.

6.2 Les quatre autres détecteurs

M2 est un CNN compact sur le spectrogramme d'amplitude (§4). **M3** est détaillé au §7. **M4** : SONICS (arXiv 2408.14080)[V] rassemble 97 164 titres (4 751 h), dont 49 074 synthétiques (Suno v2/v3/v3.5, Udio 32 et 130) et 48 090 réels de YouTube ; la généralisation chute sur Udio 32, non vu ; l'humain atteint un F1 de 0,71 ; le découpage entraînement/validation/test se fait par paroles et style. Le dépôt `awsaf49/sonics` (code MIT, données CC BY-NC 4.0) ne contient que les titres synthétiques : les réels se retrouvent via les identifiants YouTube de `real_songs.csv`. **M0** : §3.

6.3 Généralisation hors distribution[N18]

Le projet entraîne sur des faux de *codec* (EnCodec, DAC, Mimi) et teste sur des générateurs *jamais vus* : on remplit une **matrice** détecteur × générateur (M1–M4 contre les cinq générateurs de FakeMusicCaps), chaque case étant une AUC avec son intervalle bootstrap groupé par légende. Quand on dispose de faux de plusieurs générateurs, la validation *leave-one-generator-out* (entraîner sur tous sauf un, tester sur celui-ci) mesure le transfert. Trois leçons de la littérature :

- **Le test i.i.d. est saturé** : Afchar et al. 99,8 %, FakeMusicCaps ≈ 1,00 en ensemble fermé (0,75–0,85 en ensemble ouvert), Park et al. AUC ≈ 99,8–99,9 % sur les découpages standards[V].
- **Le transfert dépend de la famille** : bon à l'intérieur d'une famille de décodeurs, quasi nul entre familles (Afchar) ; face à un générateur non vu, le meilleur espace de jetons change (Park : sur Udio, CLAM tombe à 66,5 % d'AUC et X-Codec monte à 89,0 %).
- **Le point de fonctionnement compte** : voir §4.

MusicDET (zéro-coup, entraîné sans générateur) montre qu'on peut s'en passer, mais s'effondre sous perturbations. L'ensemble motive les hypothèses H1–H2 du projet : prédire *avant* de mesurer quel générateur transférera.

Lire la matrice (guide d'interprétation, à confronter aux prédictions pré-enregistrées ; ce sont mes lectures, pas des résultats) :

Observation	Lecture possible
AUC élevée sur MusicGen, ≈ 0,5 sur les quatre autres	compatible avec H1–H2 : seul le décodeur EnCodec est partagé
AUC élevée partout, *y compris* M0	fuite : le détecteur lit un raccourci (H3), pas l'artefact
AUC ≈ 0,5 partout, y compris le contrôle positif	montage défaillant (chaîne, étiquettes, bande), pas un résultat scientifique
IC larges qui se recouvrent	pas de différence démontrée : ne pas classer les détecteurs
AUC haute mais TPR@1 %FPR faible sous attaque	seuil non adapté (§4) plutôt que perte d'information

6.4 Interprétabilité, analyse d'erreurs, test d'écoute[N22]

- **Lire le modèle** : pour M1, tracer les poids de la régression en fonction de la fréquence ; *(hypothèse du projet : ils présentent des pics aux multiples de f_e).* Pour M3, visualiser le filtre appris (le dépôt contient `visualize_weights.py`).
- **Analyser les erreurs** : ventiler les erreurs par genre, niveau, présence de voix, position du segment ; tirer les extraits mal classés *au hasard* dans chaque cellule (pas les plus frappants).
- **Test d'écoute** (lab L8) : 20 extraits en ordre aléatoire, réponse binaire plus confiance ; analyser taux de réussite et accord entre auditeurs. Repère : F1 humain de 0,71 sur SONICS[V].
- **Cas difficiles** : l'audio *édité* (arXiv 2608.14916)[R] produit des faux positifs (PaSST : exactitude équilibrée de 0,811 sur 933 vidéos, 95 groupes de morceaux, découpage par groupe) ; AI-OpenBMAT (arXiv 2602.06823)[V] fait chuter SpecTTTra et un CNN sous 62 % de F1 quand la musique est en arrière-plan.

7 Robustesse et invariance d'échelle

7.1 Les attaques et ce qu'elles font au spectre[N21]

Attaque	Effet sur un peigne de pics nf_e	Outil
Vitesse $\times a$	$f_e \to af_e$: peigne mis à l'échelle (translation en log)	`asetrate` puis `aresample` (ffmpeg)
Hauteur	idem, durée conservée	`rubberband` (le dépôt de Barand l'emploie)[V]
Rééchantillonnage	pics inchangés, bande coupée (anti-repliement)	`soxr`[V]
MP3 64/128 kbit/s	coupure passe-bas (§2.5), quantification	`libmp3lame`
Bruit blanc	plancher qui noie les pics fins	addition à un RSB donné

Afchar et al.[V] constatent l'effondrement sous décalage de hauteur, bruit blanc et MP3/AAC/Opus à 64 kbit/s, et la stabilité sous étirement temporel, égalisation et réverbération ; MusicDET atteint 41,8 à 44,7 % d'EER sous hauteur, bruit et MP3 à 64 kbit/s[V]; ArtifactNet utilise de l'*augmentation consciente des codecs* (WAV, MP3, AAC, Opus 128 k) mais suppose du 44,1 kHz, des codecs RVQ, et un passage de Demucs fait baisser son F1 de 0,995 à 0,959[V].

7.2 Rendre la vitesse invisible : M3[N21]

Dugelay et al. (arXiv 2607.27454)[V] traitent l'invariance à la vitesse *par construction*. Notons $p(u)$ l'empreinte spectrale en fonction de $u = \log f$. Une vitesse a donne $p_a(u) = p(u - \log a)$: une *translation*. Une corrélation croisée avec un filtre h, $c(\tau) = \sum_u p(u)\, h(u - \tau)$, est *équivariante* : $c_a(\tau) = c(\tau - \log a)$; un *max-pooling global* sur τ est alors *invariant*, $\max_\tau c_a = \max_\tau c$ (aux effets de bord et de discrétisation près).

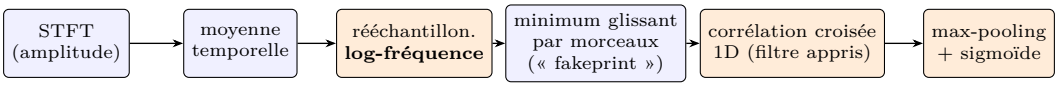


Figure 10: Chaîne de M3 d'après Dugelay et al. Les trois blocs orange portent l'invariance à la vitesse (translation en log, équivariance de la corrélation, invariance du max).

Un piège de discrétisation (mesure personnelle sur un jouet, *pas* la méthode de l'article). Un peigne de pas f_e jusqu'à $f_{\max}$ a, en haut de bande, un espacement de $\log_2(1 + f_e/f_{\max})$ octave : 0,0135 pour 75 Hz à 8 kHz. Il faut donc une grille log *fine* (de l'ordre de 150 bins par octave) pour ne pas sous-échantillonner les pics. Test : peigne de pics tous les 75 Hz sur 1–8 kHz, filtre à moyenne nulle extrait du motif, score $= \max_\tau$ de la corrélation, vitesses $\times 1$ à $\times 1{,}4$. Avec 48 bins/octave le score s'effondre dès $\times 1{,}05$ (4,8 puis 1,7 à 2,7) ; avec 192 bins/octave il reste entre 22 et 26 (contre 0,9 pour un spectre sans peigne). Pour une translation *exacte* (`np.roll`), le score est identique à six décimales ; sous une vraie vitesse, la troncature de la bande et la discrétisation le rendent seulement *approximativement* invariant.

Détails : perte hybride BCE $+ \lambda$ CE avec $\lambda = 0{,}05$, la seconde portant sur la *position du pic* de corrélation ; entraînement avec des vitesses discrètes $\times 0{,}7$ à $\times 1{,}4$, test à vitesses continues ; données : 5 000 morceaux Suno v5 et 5 000 FMA-small de 30 s, plus Suno v3.5 et Udio 120 de SONICS. Résultats : sous attaque de vitesse, AUC 0,997 et F1 0,986 sur Suno v5, contre 0,817 et 0,675 pour le détecteur d'Afchar et al. ; AUC supérieure à 0,93 même *sans* entraînement avec attaque ; pour la hauteur seule, la réussite est partielle (AUC 0,963, mais F1 0,279 : problème de seuil, §4). Limites déclarées : une seule attaque, trois générateurs ; MP3, égalisation, codecs non testés. Le dépôt (`scripts/pipeline, train, test, visualize_weights`, `research/` pour la corrélation croisée et la CQT) a deux jeux de points de contrôle, `attack` et `noattack` ; sa licence n'est pas indiquée sur la page lue.

Lien avec le projet

(hypothèse du projet : H4 : la log-fréquence résiste à la vitesse mais pas nécessairement au MP3 ni au bruit.) Pourquoi : une translation n'enlève rien au peigne, tandis que la coupure passe-bas du MP3 *supprime* des pics (§2.5) et le bruit *noie* les pics fins. Protocole : pour chaque détecteur, courbes AUC et TPR@1 %FPR en fonction de l'intensité de chaque attaque (vitesse $\times 0{,}7$ à $\times 1{,}4$; MP3 64–320 kbit/s ; RSB ; rééchantillonnage), seuil figé sur données propres, avec et sans entraînement sous attaque.

8 Données, outils, méthode et éthique

8.1 Jeux de données

FMA[V]	106 574 pistes de 16 341 artistes (917 GiB au total), *tout en MP3*. Sous-ensembles : *small* 8 000 extraits de 30 s, 8 genres équilibrés (7,2 GiB) ; *medium* 25 000, 16 genres (22 GiB) ; *large* 106 574 extraits de 30 s, 161 genres (93 GiB) ; *full* 879 GiB. Métadonnées (`fma_metadata.zip`, 342 MiB : `tracks.csv`, `genres.csv`, `features.csv`, `echonest.csv` pour 13 129 pistes) ; un découpage entraînement/validation/test est fourni. Code MIT, métadonnées CC BY 4.0, audio sous la licence de chaque artiste, usage de recherche.
FakeMusicCaps[V]	27 605 extraits de 10 s (≈ 77 h), mono 16 kHz, flottant 32 bits, WAV ; cinq générateurs ; jeu de test de 320 pistes tirées de légendes SunoCaps. Dépôt Zenodo 13732524 (v1, 8 septembre 2024), CC BY-NC 4.0, **fichiers à accès restreint** (connexion et demande) ; une version plus récente existe. La page annonce une taille de 2,9 (To ?) alors que le format décrit donne $27\,605 \times 10 \times 16\,000 \times 4$ octets ≈ 18 Go : à vérifier.
SONICS[V]	Voir §6 : titres synthétiques seuls sur Hugging Face (`awsaf49/sonics`) ou Kaggle ; réels à reconstituer depuis YouTube ; colonne `split`.

8.2 Dépôts de code et outils

Dépôt	Licence	Usage
`deezer/ismir25-ai-music-detector`[V]	CC BY-NC 4.0 ; 2 brevets	fakeprints (M1)
`deezer/deepfake-detector`[V]	CC BY-NC 4.0	détecteurs sur auto-encodeurs ; demande les poids de Musika et LAC
`thomas0barand/robust-deepfake-detector`[V]	non indiquée	M3 ; Python 3.13, `pip install -e .`
`facebookresearch/encodec`[V]	code MIT ; poids : non indiqué sur la page	codec : `EncodecModel.encodec_model_24khz()`, `set_target_bandwidth(6.0)` (8 codebooks à 6 kbit/s), `encode` renvoie $[B, n_q, T]$; ligne de commande `encodec -b 6 in.wav out.wav`[M]
`descriptinc/descript-audio-codec`[V]	code et poids MIT	`dac.utils.download(model_type="44khz")` ; `python3 -m dac encode/decode`
`awsaf49/sonics`[V]	code MIT, données CC BY-NC	M4 (SpecTTTra)
`DolbyLaboratories/neural-upsampling-artifacts-audio`[V]	MIT	notebooks sur les artefacts de suréchantillonnage

8.3 Méthode : hypothèses, ablations, contrôles, pré-enregistrement[N23]

Une *hypothèse falsifiable* s'écrit avec sa **prédiction chiffrée** et son **critère de réfutation** (ici, les conventions du §4.4). Une *ablation* modifie *un seul facteur à la fois* (représentation linéaire ou log, taille de fenêtre, bande, codec, débit) ; sinon on ne sait pas ce qui a causé l'écart. Les *contrôles* (§3) prouvent que l'instrument mesure ce qu'on croit. Le *pré-enregistrement* fige, avant les expériences et avec une date, les hypothèses, les prédictions et le plan d'analyse ; les écarts sont consignés. Une hypothèse réfutée est un résultat valide. Grille de lecture d'un article, en six questions : quelles données et quel découpage ? quelle métrique et quel seuil ? quels contrôles ? quelles limites déclarées ? quel code ? qu'est-ce qui me ferait changer d'avis ?

8.4 Mise en œuvre : squelette de M2 et pièges fréquents

Squelette de M2 (schéma pédagogique, *non exécuté ici*, PyTorch étant absent de l'environnement de rédaction ; l'architecture exacte d'Afchar et al. n'est pas reprise). Les max-poolings portent sur la *fréquence seulement* pour garder la résolution temporelle ; avec les canaux ci-dessous, on compte environ 1,1 M de paramètres (calcul à la main) :

```
class SmallCNN(nn.Module):                  # entree : [B, 1, F, T] spectrogramme d'amplitude en dB
    def __init__(self, ch=(32, 64, 128, 128, 256, 256)):
        super().__init__(); layers, c_in = [], 1
        for c in ch:
            layers += [nn.Conv2d(c_in, c, 3, padding=1), nn.BatchNorm2d(c), nn.ReLU(), nn.MaxPool2d((2, 1))]
            c_in = c
        self.features, self.head = nn.Sequential(*layers), nn.Linear(c_in, 1)
    def forward(self, x):
        return self.head(self.features(x).mean(dim=(2, 3))).squeeze(1)   # logit ; perte : BCEWithLogitsLoss
```

On tire des *segments aléatoires* dans chaque extrait de 10 s (le même nombre par morceau), on agrège les logits par morceau (moyenne) et l'on évalue au niveau du groupe d'évaluation (morceau, artiste) *en cohérence avec le bootstrap*.

Ordres de grandeur (calculs, pour dimensionner le stockage). FMA-small : $8\,000 \times 30$ s découpés en segments

de 10 s donnent 24 000 segments ; décodés en mono 16 kHz, $8\,000 \times 30 \times 16\,000 \times 2$ octets $\approx 7{,}7$ Go en entier 16 bits (15,4 Go en flottant 32 bits). Chaque variante de codec ou d'attaque multiplie ce volume : un jeu avec un réel, deux codecs et trois débits par codec fait environ 7× le volume du réel. FakeMusicCaps : $27\,605 \times 10 \times 16\,000 \times 4$ octets $\approx 17{,}7$ Go.

Pièges fréquents (conseils d'ingénierie, pas des résultats d'articles) :

- **Découper avant de segmenter** : séparer entraînement, validation et test par artiste (ou légende) *d'abord*, couper en segments ensuite ; calculer les statistiques de normalisation sur l'entraînement seul.
- **Mauvaise fréquence déclarée** : lire un fichier à 24 kHz comme s'il était à 16 kHz est un changement de vitesse ×1,5 (§2.4) ; vérifier f_s à chaque chargement.
- **Écrêtage et échelle** : un codec peut dépasser ±1 ; contrôler le pic avant d'écrire en entier 16 bits ; rester en flottant.
- **Normaliser par fichier** (moyenne et écart-type du spectrogramme) efface le niveau mais peut aussi effacer l'amplitude de l'artefact : à ablater.
- **Mode d'évaluation** : `model.eval()` (BatchNorm) ; ne jamais choisir le seuil sur le test ; ne pas régler les hyperparamètres sur les 320 pistes de test de FakeMusicCaps.
- **Rapporter tout** : la matrice complète, échecs compris, au moins trois graines, un journal de tous les essais.

8.5 Licences et éthique[N24]

Licences (ceci n'est pas un avis juridique). CC BY-NC 4.0 impose l'attribution et interdit l'usage commercial ; ne pas redistribuer de jeu dérivé hors de ces conditions. Les pistes FMA relèvent de la licence de chaque artiste : vérifier avant de republier des extraits ou des versions ré-encodées. Les dépôts de Deezer signalent des demandes de brevet : lire les conditions avant toute exploitation. **Éthique.** Un score de détecteur n'est pas une preuve : un faux positif peut accuser un artiste à tort (cas de l'audio édité). Publier des faiblesses d'un détecteur aide aussi à le contourner (double usage) ; en rendre compte avec mesure. Les faux de codec fabriqués avec de la musique d'artistes servent à l'évaluation interne et ne se diffusent pas. Un test d'écoute avec des participants demande leur consentement et l'anonymisation des réponses.

A Sources et liens

Légende. [V] ouvert (extraction) ; [R] extrait de recherche ; [N] non ouvert. Ouvert ne veut pas dire relu : relire le PDF avant de citer un chiffre.

Détection

Afchar et al., *Detecting music deepfakes...* https://arxiv.org/abs/2405.04181 [V]
Afchar et al., *A Fourier Explanation of AI-music Artifacts* https://arxiv.org/abs/2506.19108 [R]
Dugelay et al., robustesse (ISMIR 2026) https://arxiv.org/abs/2607.27454 [V]
Comanducci et al., FakeMusicCaps https://arxiv.org/abs/2409.10684 [V]
FakeMusicCaps, Zenodo https://zenodo.org/records/13732524 [V]
FakeMusicCaps, *J. Imaging* 2025 https://www.ncbi.nlm.nih.gov/pmc/articles/PMC12295441/ [N]
Han et al., MusicDET https://arxiv.org/abs/2605.18072 [V]
Oh, ArtifactNet https://arxiv.org/abs/2604.16254 [V]
Park et al., Probing Token Spaces https://arxiv.org/abs/2606.08663 [R]
Audio édité, cas difficiles https://arxiv.org/abs/2608.14916 [R]
SONICS https://arxiv.org/abs/2408.14080 [V]
AI-OpenBMAT https://arxiv.org/abs/2602.06823 [V]
Müller et al. 2022, généralisation https://arxiv.org/abs/2203.16263 [V]
Müller et al., silence https://arxiv.org/abs/2106.12914 [V]
Geirhos et al., shortcut learning https://arxiv.org/abs/2004.07780 [V]
Guo et al., calibration https://arxiv.org/abs/1706.04599 [V]
Choi et al., DL pour la MIR https://arxiv.org/abs/1709.04396 [V]

Codecs et générateurs

SoundStream https://arxiv.org/abs/2107.03312 [V]
EnCodec https://arxiv.org/abs/2210.13438 [V]
DAC https://arxiv.org/abs/2306.06546 [V]
Mimi (article Moshi) https://arxiv.org/abs/2410.00037 [V]
Codec-SUPERB https://arxiv.org/abs/2402.13071 [V]
MusicGen https://arxiv.org/abs/2306.05284 [V]
Stable Audio Open https://arxiv.org/abs/2407.14358 [V]
Mustango https://arxiv.org/abs/2311.08355 [V]
AudioLDM 2 https://arxiv.org/abs/2308.05734 [V]
MusicLDM https://arxiv.org/abs/2308.01546 [V]
Musika https://arxiv.org/abs/2208.08706 [V]
AST https://arxiv.org/abs/2104.01778 [R]
Pons et al., artefacts de suréchantillonnage https://arxiv.org/abs/2010.14356 [R]
Odena et al., damier (Distill) https://distill.pub/2016/deconv-checkerboard/ [V]
arXiv 2402.13236, 2406.00146, 2603.09936 [N]

Données et dépôts

FMA https://github.com/mdeff/fma [V]
Deezer, fakeprints https://github.com/deezer/ismir25-ai-music-detector [V]
Deezer, détecteurs AE https://github.com/deezer/deepfake-detector [V]
Barand, M3 https://github.com/thomas0barand/robust-deepfake-detector [V]
EnCodec https://github.com/facebookresearch/encodec [V]
DAC https://github.com/descriptinc/descript-audio-codec [V]

SONICS https://github.com/awsaf49/sonics [V]
AI-OpenBMAT https://github.com/DaveLoay/AI-OpenBMAT [R]
Dolby, artefacts https://github.com/DolbyLaboratories/neural-upsampling-artifacts-audio [V]
dl4mir (Choi) https://github.com/keunwoochoi/dl4mir [N]

Cours et pédagogie

MVA, G. Richard (slides) https://perso.telecom-paristech.fr/grichard/Enseignements/MVA/2026-MVA_indexation-english.pdf [V]
FMP, M. Müller https://www.audiolabs-erlangen.de/resources/MIR/FMP/C2/C2.html [V]
J. O. Smith, SASP https://ccrma.stanford.edu/~jos/sasp/ [V]
Hugging Face Audio Course https://huggingface.co/learn/audio-course [V]
Weng, VAE https://lilianweng.github.io/posts/2018-08-12-vae/ [V]
Weng, diffusion https://lilianweng.github.io/posts/2021-07-11-diffusion-models [V]
CS236, notes https://deepgenerativemodels.github.io/notes/index.html [V]
AssemblyAI, RVQ https://assemblyai.com/blog/what-is-residual-vector-quantization [V]
Forasoft, MP3 https://www.forasoft.com/learn/audio-for-video/articles-audio/how-audio-compression-works [V]
MP3 (mémoire, typeset) https://typeset.io/pdf/the-theory-behind-mp3-4a7suq9ncw.pdf [N]
CS231n http://cs231n.stanford.edu [N]
CS224S https://web.stanford.edu/class/cs224s/index.html [N]
Velardo, *The Sound of AI* https://www.youtube.com/watch?v=iCwMQJnKk2c&list=PL-wATfeyAMNqIee7cH3q1bh4QJFAaeNv0 [N]
Velardo, PyTorch pour l'audio https://www.youtube.com/watch?v=gp2wZqDoJ1Y&list=PL-wATfeyAMNoirN4idjev6aRu8ISZYVWm [N]

B Exercices et corrigés

1. **(N1)** Un sinus à 30 kHz est échantillonné à 44,1 kHz. Où apparaît-il ? *R. :* $44{,}1 - 30 = 14{,}1$ *kHz.*
2. **(N2)** À $f_s = 16$ kHz, $N = 2\,048$: résolution et durée de fenêtre ? *R. :* $16\,000/2\,048 = 7{,}8$ *Hz ;* 128 *ms.*
3. **(N3)** De combien de demi-tons la 3e harmonique dépasse-t-elle la 2e, quelle que soit f_0 ? *R. :* $12\log_2 1{,}5 \approx 7{,}02$ *(une quinte).*
4. **(N4)** Vitesse ×0,8 : nouvelle durée, décalage en demi-tons, nouvel espacement des pics d'EnCodec ? *R. : durée* ×1,25 *;* $12\log_2 0{,}8 \approx -3{,}86$ *;* $75 \to 60$ *Hz.*
5. **(N5)** Avec l'ancrage à 16 kHz, la coupure d'un MP3 stéréo à 64 kbit/s (11 kHz mesurés) se voit-elle ? *R. : non, elle est au-dessus de 8 kHz ; l'attaque MP3 agit par la quantification.*
6. **(N12)** Motif de recouvrement pour $K = 5$, $s = 3$ et pour $K = 6$, $s = 3$? *R. :* $1, 2, 2$ *répété ; uniforme (2) car* K *multiple de* s.
7. **(N25)** Strides $(4, 4, 2)$ en sortie à 16 kHz : cadence d'entrée et nombre de pics ? *R. :* $P_{\max} = 32$, $f_e = 500$ *Hz,* $\lfloor 32/2 \rfloor + 1 = 17$ *pics (0 à 8 kHz).*
8. **(N13)** Débit de MusicGen (50 Hz, 4 codebooks, $K = 2\,048$) ? N_q pour EnCodec 24 kHz à 12 kbit/s ? *R. :* $50 \times 4 \times 11 = 2{,}2$ *kbit/s ;* $12\,000/(75 \times 10) = 16$.
9. **(N6)** Champ réceptif de conv. 3 (pas 1), conv. 3 (pas 2), conv. 3 (pas 2) ? *R. :* $3 \to 5 \to 9$.
10. **(N8)** Faux $\{0{,}95; 0{,}6; 0{,}55\}$, réels $\{0{,}7; 0{,}5; 0{,}1\}$: AUC et EER ? *R. :* $7/9$ *;* $1/3$.
11. **(N26)** Cent extraits proviennent de 10 artistes. Pourquoi le bootstrap sur les extraits est-il trop optimiste ? *R. : les extraits d'un artiste sont corrélés ; la taille utile est proche de 10, pas 100.*
12. **(N19–20)** M0 donne AUC 0,70 (IC 0,66–0,74) après la chaîne. Que faire ? *R. : il reste un indice ; identifier la caractéristique (importance, ablation), corriger la chaîne, refaire ; sinon, le rapporter comme limite.*

C Laboratoires et parcours

Lab	Contenu (CPU, 0,5 à 2 h)	À observer
L1 Aliasing	sinus près de Nyquist, rééchantillonnage sans puis avec filtre, STFT	répliques et repliement sans filtre
L2 MP3	extrait FMA en MP3 à plusieurs débits (`ffmpeg`), spectre moyen	coupure selon débit et mode ; *refaire sur de la vraie musique*
L3 Vitesse	son harmonique, vitesse ×0,9 et ×1,1, axe linéaire puis log	sur l'axe log : translation
L4 Pics	conv. transposée contre plus proche voisin puis conv., poids aléatoires, **entrée lisse** (pas du bruit blanc), spectre moyen	figure 7
L5 Codec	extrait dans EnCodec puis DAC, spectres moyens, débits variés (GPU ou Colab)	pics ou coupure selon le débit
L6 M0	descripteurs de fichier, classifieur, AUC avec IC groupé	AUC $> 0{,}5$ avant d'écouter : raccourci
L7 Métriques	ROC, AUC, EER, TPR@1 %FPR en NumPy, bootstrap simple, groupé, apparié	les incertitudes diffèrent
L8 Écoute	protocole 20 extraits, script d'analyse	protocole prêt avant de recruter
L9 Chaîne	branches réel et faux identiques sauf le codec ; spectres, niveaux, alignement ; M0 résiduel	plus d'indice trivial

Correction au protocole de L4 : le dossier prévoyait un bruit blanc en entrée ; la figure 7c montre qu'il ne fait pas apparaître de pics (entrée lisse requise). **Ajout** (L3b) : refaire le test de discrétisation du §7 (48 contre 192 bins par octave).

Parcours. Durées d'apprentissage du dossier (estimations personnelles, non mesurées, parcours approfondi) ; la lecture de ce cours seul représente environ 8 h.

Phase	Dans ce cours	Articles et pages à ouvrir (dans l'ordre)	Labs
Avant S1 (10–12 h)	§2, §3.1–2	MVA §2 ; HF Audio, unité sur les données audio ; FMP, chapitre 2 (option) ; Velardo (vidéos)	L1, L3
S1 (8–10 h)	§5.2, §2.5, §6.1	Fourier (2 lectures) ; Odena (30 min) ; Pons ; Müller 2022 ; Afchar 2024 (1re lecture) ; MVA §3	L2, L4, L9
S2 (8–10 h)	§5 (5.1, 5.3, 5.4)	AssemblyAI (30 min) ; SoundStream (partiel) ; EnCodec (complet) ; DAC (résumé, méthode, ablations) ; MusicGen (option)	L5, L9
S3 (8–10 h)	§3, §4, §6	Geirhos (introduction) ; Müller (silence) ; Afchar 2024 (complet) ; FakeMusicCaps ; Guo ; Choi ; MVA §7–8	L6, L7
S4 (6–8 h)	§7, §8	Dugelay (complet) ; SONICS (SpecTTTra, cadre d'évaluation) ; AST (option) ; MusicDET, ArtifactNet, Park (extension)	L3 relu, L8

Comment lire quelques articles (dans les sections indiquées par leur titre, non par leur numéro) :

Article	Chercher	Garder en tête
Afchar 2024	tableaux de généralisation entre décodeurs ; courbes de robustesse	99,8 % en distribution ne dit rien du transfert
Fourier (2025)	formule des pics ; recette de l'empreinte ; résultats Udio 32	dépend de l'architecture ; interpolation non démontrée
Dugelay (2026)	pipeline ; perte hybride ; tableaux sous attaque	une attaque, trois générateurs
FakeMusicCaps	format ; ensembles fermé et ouvert	résultats préliminaires ; test de 320 pistes
EnCodec, DAC	encodeur, RVQ, pertes ; débits ; ablations	le décodeur est la source de l'artefact
Geirhos	taxonomie des règles de décision	« stratégie ≠ intention »
Müller (silence)	tableaux avec et sans silence	précédent exact de H3 et de M0
Guo	ECE, *temperature scaling*	ne change pas l'AUC

D Points à revérifier et limites

- **MusicDET** : l'EER du meilleur détecteur discriminatif comparé a été lu 11,46 %, 15,64 % et 17,63 % selon les lectures (tableaux différents ?) ; rouvrir l'article avant de citer.
- **FakeMusicCaps** : version longue (PMC12295441) *non lue* (page protégée par un contrôle anti-robot) : coller le texte ou le PDF pour l'intégrer ; archive Zenodo à accès restreint, présence des extraits réels de MusicCaps et taille réelle non confirmées.
- **Non ouverts** : arXiv 2402.13236, 2406.00146, 2603.09936 (non identifié) ; PDF du MP3 (typeset), CS231n, CS224S, dépôt dl4mir ; vidéos de Velardo (le cours en couvre le contenu).
- **Générateurs** : décodeur de Mustango et variante de HiFi-GAN d'AudioLDM 2 non précisés ; fréquences natives de MusicLDM et d'AudioLDM 2 (16 kHz) d'après leurs articles ; AST (log-mel, patchs, initialisation) de mémoire.
- **Licences** : poids d'EnCodec et dépôt de Barand non précisés sur les pages lues.
- **Mes calculs et mesures** : nombre de pics par décodeur (hypothèse d'un suréchantillonnage par insertion de zéros) ; coupures MP3 (bruit blanc, ffmpeg 6.1.1) ; figure 7 (jouet). Ils illustrent ; ils ne remplacent pas les articles.
- **Fourier (2506.19108)** : bandes exactes et résultats à relire dans le PDF. **FMA** : vérifier si le découpage fourni sépare les artistes (colonne `(set, split)` de `tracks.csv`).

E Aide-mémoire des formules

Nyquist, répliques	$f_N = f_s/2$; $X_s(f) = f_s \sum_k X_c(f - kf_s)$; repliement $f_0 \mapsto \lvert f_0 - kf_s\rvert$
Quantification	RSB $\approx 6{,}02\,b + 1{,}76$ dB (sinus pleine échelle)
DFT, STFT	$X[k] = \sum_n x[n] e^{-2\pi i k n/N}$; $f_k = kf_s/N$; $\Delta f = f_s/N$; $X[m,k] = \sum_n x[n+mH]w[n]e^{-2\pi i k n/N}$
Échelles	$\mathrm{dB} = 20\log_{10}\lvert X\rvert$; $\mathrm{mel} = 2595\log_{10}(1 + f/700)$; CQT : $f_k = f_{\min} 2^{k/B}$, $Q = 1/(2^{1/B} - 1)$
Vitesse a	$f' = af$, durée $/a$; $\log f' = \log f + \log a$; décalage $= 12\log_2 a$ demi-tons
Insertion de zéros	$u[n] = s[n/k]$; $U(e^{i\omega}) = S(e^{ik\omega})$; pics en nf_e, $n \le \lfloor k/2\rfloor$; $P_{\max} = \prod k_i$, $\lfloor P_{\max}/2\rfloor + 1$ pics
Conv. transposée	longueur $(L-1)s + K$; recouvrement uniforme ssi $s \mid K$
Champ réceptif	$r_l = r_{l-1} + (k_l - 1)j_{l-1}$; $j_l = j_{l-1}s_l$
VAE	$\mathbb{E}_q[\log p(x\lvert z)] - \mathrm{KL}(q(z\lvert x)\Vert p(z))$; $z = \mu + \sigma \odot \varepsilon$
Diffusion	$x_t = \sqrt{\bar\alpha_t}x_0 + \sqrt{1-\bar\alpha_t}\,\varepsilon$; $L = \mathbb{E}\Vert\varepsilon - \varepsilon_\theta(x_t,t)\Vert^2$; $\tilde\varepsilon = (w+1)\varepsilon(x,c) - w\varepsilon(x)$
VQ, RVQ, débit	$q(z) = c_{\arg\min_j \Vert z - c_j\Vert}$; $r_q = r_{q-1} - c^q_{k_q}$; $R = f_{\mathrm{trame}} N_q \log_2 K$
AUC, EER	$\mathrm{AUC} = \frac{1}{n^+n^-}\sum \mathbf{1}[s^+ > s^-]$; EER : FPR $= 1 -$ TPR
Calibration	$\mathrm{ECE} = \sum_m \frac{\lvert B_m\rvert}{n}\lvert\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\rvert$; temperature scaling : $\mathrm{softmax}(z/T)$
Statistique	IC bootstrap groupé : centiles 2,5 et 97,5 de $\mathrm{AUC}^{(b)}$; Holm : rejeter $H_{(i)}$ si $p_{(i)} \le \alpha/(m-i+1)$
Invariance	$c(\tau) = \sum_u p(u)h(u-\tau)$; $p_a(u) = p(u - \log a) \Rightarrow c_a(\tau) = c(\tau - \log a)$; $\max_\tau c_a = \max_\tau c$

F Glossaire (français / anglais)

Fréquence d'échantillonnage (*sample rate*) : échantillons par seconde ; fixe la bande utile.

Repliement (*aliasing*) : fréquences au-dessus de Nyquist repliées dans la bande.

STFT : spectre sur fenêtres glissantes ; **hop** : pas entre fenêtres.

Fuite spectrale (*leakage*) : étalement d'une raie dû à la fenêtre.

Mel : échelle perceptive de fréquence ; **CQT** : transformée à Q constant (bins log).

Rééchantillonnage (*resampling*) : changer f_s sans changer le contenu en Hz.

Ancrage : fréquence commune imposée à toutes les données (16 kHz ici).

Changement de vitesse : hauteur et durée changent ensemble.

Étirement (*time stretch*), **décalage de hauteur** (*pitch shift*) : durée ou hauteur seule.

MDCT, **masquage psychoacoustique** : briques du MP3.

Codec neuronal : encodeur–décodeur appris à goulot quantifié.

Codebook : dictionnaire de vecteurs ; **RVQ** : quantification résiduelle à plusieurs dictionnaires.

Débit (*bitrate*) : bits par seconde ; **jeton** (*token*) : indice de dictionnaire.

Vocodeur : réseau mel → onde (HiFi-GAN).

Convolution transposée : suréchantillonnage appris ; peut laisser un damier (*checkerboard*).

Fakeprint : empreinte spectrale moyenne d'un générateur.

Décodeur oracle : décodeur du générateur visé, utilisé pour fabriquer des faux.

AUC, **EER**, TPR@1 %FPR : voir §4.

Calibration : probabilités prédites = fréquences observées ; **ECE** : écart moyen.

Bootstrap groupé : rééchantillonnage par groupes (artiste, légende).

Test apparié : deux détecteurs comparés sur les mêmes groupes.

Raccourci (*shortcut*) : signal corrélé à l'étiquette sans rapport avec la tâche.

Fuite (*leakage*) : information de test dans l'entraînement.

Contrôle négatif / positif : expérience censée donner un résultat nul / maximal.

Pré-enregistrement : hypothèses et plan d'analyse figés avant les expériences.

Hors distribution (*OOD*) : données différentes de l'entraînement.

Ensemble ouvert (*open set*) : le test contient des classes jamais vues.

Zéro-coup (*zero-shot*) : sans exemple de la classe à détecter.